

PSP Manual Vacuum on Amazon RDS for PostgreSQL

How Does Vacuum work in PostgreSQL:

VACUUM reclaims storage occupied by dead tuples. In normal PostgreSQL operation, tuples that are deleted or obsoleted by an update are not physically removed from their table, they remain present until a VACUUM is done. Therefore it's necessary to do VACUUM periodically, especially on frequently-updated tables.

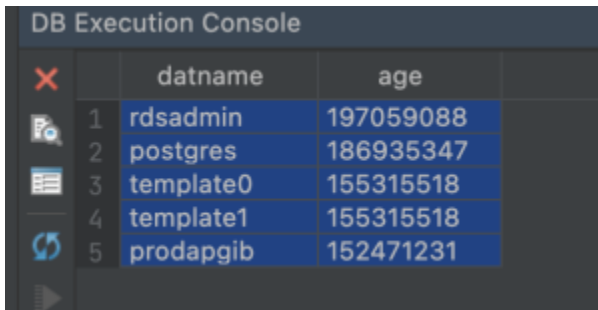
- **database need vacuuming**

The column **datfrozenxid** of a database's **pg_database** row is a lower bound on the normal transaction IDs appearing in that database. This column is the minimum of the per-table **relfrozenxid** values within the database.

show the number of unvacuumed transactions in a database.

```
SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY age(datfrozenxid) desc limit 20;
```

Example , Result:



	datname	age
1	rdsadmin	197059088
2	postgres	186935347
3	template0	155315518
4	template1	155315518
5	prodapgib	152471231

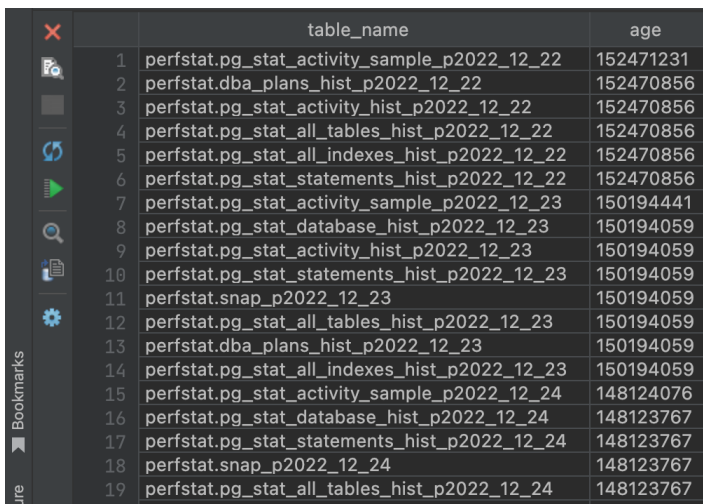
If database age reaches 145M transaction IDs, we need to do manual vacuum.

- **which tables are need vacuuming**

Tables whose relfrozenxid value is autovacuum_freeze_max_age greater than the number of transactions in are always targeted by vacuum. Otherwise, if the number of tuples made obsolete since the last VACUUM exceeds the vacuum threshold, the table is vacuumed.

```
SELECT c.oid::regclass as table_name,  
       greatest(age(c.relfrozenxid),age(t.relfrozenxid)) as age  
FROM pg_class c  
     LEFT JOIN pg_class t ON c.reltoastrelid = t.oid  
WHERE c.relkind IN ('r', 'm') order by 2 desc;
```

Example, Result:



	table_name	age
1	perfstat.pg_stat_activity_sample_p2022_12_22	152471231
2	perfstat.dba_plans_hist_p2022_12_22	152470856
3	perfstat.pg_stat_activity_hist_p2022_12_22	152470856
4	perfstat.pg_stat_all_tables_hist_p2022_12_22	152470856
5	perfstat.pg_stat_all_indexes_hist_p2022_12_22	152470856
6	perfstat.pg_stat_statements_hist_p2022_12_22	152470856
7	perfstat.pg_stat_activity_sample_p2022_12_23	150194441
8	perfstat.pg_stat_database_hist_p2022_12_23	150194059
9	perfstat.pg_stat_activity_hist_p2022_12_23	150194059
10	perfstat.pg_stat_statements_hist_p2022_12_23	150194059
11	perfstat.snap_p2022_12_23	150194059
12	perfstat.pg_stat_all_tables_hist_p2022_12_23	150194059
13	perfstat.dba_plans_hist_p2022_12_23	150194059
14	perfstat.pg_stat_all_indexes_hist_p2022_12_23	150194059
15	perfstat.pg_stat_activity_sample_p2022_12_24	148124076
16	perfstat.pg_stat_database_hist_p2022_12_24	148123767
17	perfstat.pg_stat_statements_hist_p2022_12_24	148123767
18	perfstat.snap_p2022_12_24	148123767
19	perfstat.pg_stat_all_tables_hist_p2022_12_24	148123767

selected above 145M age tables and run manual vacuum.

```

\timing
set search_path to ibobadm;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m062022_from_as400;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m072022_from_psp;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m072022_from_dflt;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m082022_from_dflt;
SELECT pg_sleep(60);
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m082022_from_ews;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m082022_from_null;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m082022_from_psp;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m062022_from_cris;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m062022_from_null;
vacuum (analyze, verbose) ibobadm.psp_source_system_transmission_m062022_from_qbdt;

```

VERBOSE: The VERBOSE request prints a thorough report on the vacuum action for every table.

ANALYZE: The ANALYZE request updates the facts/statistics that the planner uses to decide the best and the most efficient way to carry out a query.

The **VACUUM ANALYZE** function, first, will process a VACUUM request, then an ANALYZE request for each of the selected tables. A simple VACUUM request will only reclaim storage, make it available for re-use.

Output:

```

Timing is on.
SET
Time: 1.066 ms
psql:run_vacuum_20March2023_10.sql:3: INFO:  aggressively vacuuming "ibobadm.
psp_source_system_transmission_m062022_from_as400"
psql:run_vacuum_20March2023_10.sql:3: INFO:  table "psp_source_system_transmission_m062022_from_as400": found 0
removable, 0 nonremovable row versions in 0 out of 0 pages
DETAIL:  0 dead row versions cannot be removed yet, oldest xmin: 538035517
Skipped 0 pages due to buffer pins, 0 frozen pages.
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s.
psql:run_vacuum_20March2023_10.sql:3: INFO:  aggressively vacuuming "pg_toast.pg_toast_568047941"
psql:run_vacuum_20March2023_10.sql:3: INFO:  table "

": found 0 removable, 0 nonremovable row versions in 0 out of 0 pages
DETAIL:  0 dead row versions cannot be removed yet, oldest xmin: 538035517
Skipped 0 pages due to buffer pins, 0 frozen pages.
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s.
psql:run_vacuum_20March2023_10.sql:3: INFO:  analyzing "ibobadm.
psp_source_system_transmission_m062022_from_as400"
psql:run_vacuum_20March2023_10.sql:3: INFO:  "psp_source_system_transmission_m062022_from_as400": scanned 0 of
0 pages, containing 0 live rows and 0 dead rows; 0 rows in sample, 0 estimated total rows
VACUUM
Time: 23.167 ms

```

Toast is a mechanism in PostgreSQL to handle large chunks of data to fit in page buffer. When the data exceeds *TOAST_TUPLE_THRESHOLD* (2KB default), Postgres will compress the data, trying to fit in 2KB buffer size. If the compressing of the large column data does not lead to smaller block (<2KB), it will be split into smaller chunks.

This is enabled by default and all tables will have the toast table associated with it. You can check the associated toast table in `pg_class` using the following query:

```

select relname from pg_class where oid = (select reltoastrelid from pg_class where relname='TABLE_NAME');

```

The table name will be `pg_toast_$(OID)` where `oid` is the toast table oid, `reltoastrelid` of original table.

The toast table is in `pg_toast` schema, so to query you need to use:

```

select * from pg_toast.pg_toast_$(OID);

```

The above query will print the toasted `chunk_id`, `chunk_seq`, `chunk_data`, if present.

Note: `pg_toast_*` table entry is only for data which is split into multiple chunks.

